Md Anas

09772515271 | mdanasakbari@gmail.com | GitHub | LinkedIn | LeetCode

Summary

Pre-final year B.Tech student interested in AI systems, Agentic AI, and Large Language Models. Passionate about studying research papers and understanding AI from first principles, with hands-on experience translating research into scalable systems — spanning LLM fine-tuning, diffusion-based generative modeling, autonomous agent frameworks, and production MLOps pipelines.

Skills

Machine Learning: Supervised & Unsupervised Learning, XGBoost, LightGBM, Feature Engineering
Deep Learning: CNN, ANN, Diffusion Models, TensorFlow, Keras, PyTorch, Deep Generative Modeling
NLP & GenAI: Hugging Face Transformers, PEFT, TRL, LoRA, QLoRA, SFT, BPE Tokenization, LangChain, Agentic AI, RAG, Prompt Engineering
MLOps & Tools: MLflow, Docker, Docker Compose, FastAPI, Streamlit, Gradio, Hugging Face Accelerate, Pydantic

Projects

FlashAttention Reimplementation | *Python, PyTorch, CUDA, Triton*

- Reimplemented the core **FlashAttention** algorithm from the paper, focusing on IO-aware attention and reducing memory traffic between **GPU HBM and SRAM**.
- Implemented **online softmax** with tiling to compute exact attention without materializing the full $N \times N$ attention matrix, reducing memory complexity from $O(N^2)$ to $O(N)$.
- Studied computational and memory bottlenecks of standard attention and analyzed how **tiling, kernel fusion, and recomputation** improve GPU memory efficiency.
- Benchmarked the implementation against standard PyTorch attention across different **sequence lengths and batch sizes**, analyzing latency, throughput, and memory usage.

Mini LLM Inference Server | *Python, PyTorch, Transformers, FastAPI, Docker*

- Built a lightweight LLM inference engine inspired by **vLLM**, implementing KV-cache management, paged attention memory allocation, continuous batching, request scheduling, sampling, and streaming inference.
- Designed a **KV-cache memory subsystem** with block-based allocation and reuse to address the memory bottleneck of autoregressive decoding.
- Implemented a **continuous batching scheduler** for dynamically admitting and advancing concurrent requests, optimizing the latency–throughput trade-off.
- Developed a benchmarking harness measuring **TTFT, inter-token latency, end-to-end latency, throughput, and memory utilization** across varying workloads and sequence lengths.
- Built modular inference and serving components with **unit/integration tests**, enabling controlled experiments with memory allocation and scheduling strategies.

Diffusion Language Model from Scratch | *Python, PyTorch, Hugging Face Accelerate, Transformers*

- Built a Diffusion Language Model **(Diffusion LM)** from scratch in PyTorch, implementing a Transformer-based **denoising architecture** with token, positional, and **timestep embeddings** to generate text through iterative diffusion-based decoding.
- Developed a complete training pipeline on the TinyStories dataset, including custom **BPE tokenization**, streaming token-block data loading, diffusion **noise scheduling**, and mixed-precision training with Hugging Face Accelerate.
- Engineered the full inference workflow with iterative **denoising** and text generation, along with visualization utilities to monitor the **diffusion process** and training progress.

Education

Government Engineering College, Ajmer — Expected June 2027
Bachelor of Technology (B.Tech) — Ajmer, Rajasthan

- **Relevant Coursework:** Data Structures & Algorithms, Operating Systems, Computer Networks, Database Management Systems, Object Oriented Programming